Projet : Étape 2

Application de Batch et Streaming

Sujet E : Analyse des avis et commentaires

Technologie	Apache Spark 3.5 (Java, Maven) : Spark Core pour le Batch, Spark Streaming pour le Streaming
Membres du groupe	[Nom Prénom 1], [Nom Prénom 2], [Nom Prénom 3] : à compléter
Enseignant(e)	[Nom de l'enseignante] : à compléter
Date	[Date de rendu] : à compléter

Contenu du document : 1. Description du besoin et choix du projet - 2. Fiche de conception - 3. Jeu de données Batch et événement Streaming - 4. Traitement Batch et résultat vérifiable - 5. Traitement Streaming et démonstration - 6. Questions de compréhension - 7. Captures, explication personnelle et difficultés - Annexe : données complètes.

1. Description du besoin et choix du projet

Nous avons choisi le sujet **E : Analyse des avis et commentaires**. Une entreprise de services (restaurant, hôtel, application mobile) reçoit en continu des avis de clients, avec une note de 1 à 5 et un commentaire libre. Le responsable qualité veut répondre à deux besoins complémentaires :

- **Comprendre l'ensemble des avis passés** : combien d'avis par catégorie, quelle note moyenne, quelle proportion d'avis positifs ou négatifs, quels mots reviennent le plus souvent dans les commentaires. C'est le rôle du **Batch**.
- **Réagir rapidement aux nouveaux avis** : voir les compteurs évoluer à chaque avis reçu et être alerté dès qu'un avis négatif arrive. C'est le rôle du **Streaming**.

Les deux traitements portent sur le **même besoin métier** (la satisfaction des clients) et utilisent les mêmes règles de calcul (même découpage des mots, mêmes catégories, même définition du sentiment), ce qui permet de comparer leurs résultats. Le Streaming ne relance pas le Batch : il traite uniquement les nouveaux avis, au fur et à mesure de leur arrivée.

2. Fiche de conception de notre projet

Rubrique	Notre réponse
Titre du projet et besoin traité	Analyse des avis et commentaires clients : comprendre les avis historiques (Batch) et suivre les nouveaux avis en direct (Streaming).
Qui utilisera les résultats ? Quel problème ?	Le responsable qualité / service client. Le projet l'aide à repérer les catégories les moins bien notées, les sujets récurrents dans les commentaires, et à réagir vite aux avis négatifs.
Données du Batch : origine, format, champs, exemple	Origine : export d'avis clients (fichier fourni dans le projet, *data/avis.csv*, 20 avis). Format : CSV avec en-tête. Champs : *id, date, categorie, note, commentaire*. Exemple de ligne : *3,2026-10-02,Hotel,2,Chambre sale et personnel pas aimable*
Traitement Batch choisi et calcul	**Spark** (API RDD en Java). Calculs : nombre total d'avis, note moyenne globale, nombre d'avis et note moyenne par catégorie, répartition des sentiments (positif : note 4-5, neutre : note 3, négatif : note 1-2), et comptage de mots (word count) pour obtenir le top 10 des mots des commentaires.
Résultat Batch attendu	Un rapport texte *output/batch_resultats.txt* affiché aussi dans la console : tableau par catégorie, répartition des sentiments, classement des 10 mots les plus fréquents.
Données du Streaming : événements et génération	Chaque événement est un nouvel avis envoyé sous la forme *categorie;note;commentaire* (ex. *Hotel;2;Chambre sale et bruit*). Ils sont générés par le programme *EventSender* (serveur socket sur le port 9999, remplace *nc -lk 9999* du TP) : un avis toutes les 2 secondes, ou saisie manuelle en mode « manuel ».
Traitement Streaming et indicateur mis à jour	**Spark Streaming** (DStream, *socketTextStream*, micro-lots de 5 secondes). Indicateurs cumulés mis à jour à chaque lot : nombre d'avis et note moyenne par catégorie, top des mots, et une **alerte** pour chaque avis dont la note est inférieure ou égale à 2.
Affichage / enregistrement des résultats	Affichage dans la console à chaque lot, et enregistrement du dernier état dans *output/streaming_resultats.txt* (Batch : *output/batch_resultats.txt*).
Comment démontrer que le Batch fonctionne ?	Test sur le petit fichier de 20 avis dont les résultats ont été calculés indépendamment (voir section 4) : on lance *BatchAvis* et on compare le rapport au résultat attendu (20 avis, moyenne 3,35, etc.).

Rubrique	Notre réponse
Comment démontrer que le Streaming fonctionne ?	On lance *EventSender* puis *StreamingAvis*. On envoie par exemple *Hotel;2;Chambre sale et bruit* et on vérifie : le compteur Hotel passe à 1, la note moyenne Hotel est 2,00 et une ligne « ALERTE avis negatif » apparaît. On envoie ensuite *Hotel;5;...* : le compteur passe à 2 et la moyenne à 3,50.
Répartition des tâches et contribution de chaque membre	[À compléter avec les vrais noms] Exemple : Membre 1 : traitement Batch (BatchAvis) et jeu de données ; Membre 2 : traitement Streaming (StreamingAvis) et générateur d'événements (EventSender) ; Membre 3 : tests de vérification, captures d'écran et rédaction du compte rendu. Chaque membre explique à l'oral la partie qu'il a codée.

3. Jeu de données Batch et exemple d'événement Streaming

3.1 Données Batch (historique)

Fichier **data/avis.csv** : 20 avis répartis en 3 catégories (Restaurant 7, Hotel 7, Application 6) pour la période du 1er au 10 octobre 2026. Extrait (le fichier complet est en annexe) :

id	date	categorie	note	commentaire
1	2026-10-01	Restaurant	5	Service rapide et plats delicieux
3	2026-10-02	Hotel	2	Chambre sale et personnel pas aimable
5	2026-10-03	Application	1	Application lente et bugs partout
...	...	...	...	...

3.2 Événements Streaming (nouveaux avis)

Format : **categorie;note;commentaire**. Les 6 événements envoyés automatiquement par *EventSender* :

N°	Événement envoyé	Alerte ?
1	Restaurant;5;Plats delicieux et service excellent	non
2	Hotel;2;Chambre sale et bruit	**oui** (note 2)
3	Application;4;Application rapide et claire	non
4	Restaurant;1;Attente longue et plats froids	**oui** (note 1)
5	Hotel;5;Personnel aimable et chambre propre	non
6	Application;1;Bugs partout application lente	**oui** (note 1)

Différence entre les deux jeux : le fichier CSV est complet, fini et relu en entier à chaque exécution ; les événements du Streaming arrivent un par un, sans fin connue, et ne sont pas stockés dans le CSV.

4. Traitement Batch (Spark) et résultat vérifiable

4.1 Principe : MapReduce avec Spark

Le Batch lit tout le fichier en une fois. Chaque calcul suit le schéma **map** (transformer chaque ligne en couple *(clé, valeur)*) puis **reduceByKey** (additionner les valeurs ayant la même clé). C'est le même principe que le word count du TP.

4.2 Étapes du programme BatchAvis.java

Étape	Instruction Spark	Explication
1. Lecture	sc.textFile(fichier)	Chaque ligne du CSV devient un élément du RDD.

Étape	Instruction Spark	Explication
2. Nettoyage	filter(...) puis map(l -> l.split(",", 5))	On retire l'en-tête et les lignes vides, puis on découpe en 5 colonnes (limite 5 : le commentaire peut contenir des virgules).
3. Avis par catégorie	mapToPair(a -> (categorie, 1)) puis reduceByKey(somme)	Map : un « 1 » par avis, clé = catégorie. Reduce : on additionne les 1 de chaque catégorie.
4. Note moyenne	mapToPair(a -> (categorie, (note, 1))) puis reduceByKey	On additionne en parallèle la somme des notes et le nombre d'avis ; moyenne = somme / nombre.
5. Sentiments	mapToPair(a -> (sentiment(note), 1)) puis reduceByKey	Note 4-5 : positif, 3 : neutre, 1-2 : négatif.
6. Word count	flatMap(mots du commentaire) puis mapToPair((mot,1)) puis reduceByKey	flatMap transforme un commentaire en plusieurs mots (minuscules, lettres seulement, 3 lettres minimum, mots vides retirés).
7. Classement	tri dans Java : effectif décroissant puis ordre alphabétique	Le résultat est petit, donc trié côté programme ; l'ordre alphabétique rend le classement reproductible.
8. Sortie	affichage + Files.write(output/batch_resultats.txt)	Le rapport est affiché et enregistré.

4.3 Test de vérification et résultat attendu

Pour vérifier, nous utilisons le fichier de 20 avis dont les résultats ont été recalculés indépendamment (par un second programme de contrôle). Le Batch Spark doit afficher exactement :

Indicateur	Résultat attendu
Nombre total d'avis	20
Note moyenne globale	3,35 (somme des notes = 67)
Application	6 avis, note moyenne 3,17
Hotel	7 avis, note moyenne 3,43
Restaurant	7 avis, note moyenne 3,43
Sentiments	positif : 11, neutre : 3, négatif : 6
Top 10 des mots	1. chambre 7 - 2. application 6 - 3. plats 6 - 4. rapide 6 - 5. service 6 - 6. aimable 5 - 7. personnel 5 - 8. interface 4 - 9. claire 3 - 10. delicieux 3

Contrôle de cohérence : 6 + 7 + 7 = 20 avis ; 11 + 3 + 6 = 20 avis. Si le rapport obtenu correspond à ce tableau, le Batch est validé.

5. Traitement Streaming (Spark Streaming) et démonstration

5.1 Principe

Spark Streaming découpe le flux en **micro-lots** de 5 secondes. Pour chaque lot non vide, le programme calcule les compteurs du lot (même schéma map / reduceByKey que le Batch) puis les **ajoute à un état cumulé** conservé depuis le démarrage. Seuls les nouveaux avis sont traités : l'historique n'est jamais relu.

5.2 Étapes du programme StreamingAvis.java

Étape	Instruction	Explication
1. Contexte	new JavaStreamingContext(conf, Durations.seconds(5)) avec master local[2]	Micro-lots de 5 s. Deux threads minimum : un pour recevoir les données, un pour les traiter.
2. Source	ssc.socketTextStream("localhost", 9999)	Chaque ligne reçue sur le socket est un avis (comme le socket du TP).
3. Validation	map(parseLigne) puis filter(nonNull)	On vérifie 3 champs séparés par « ; » et une note entre 1 et 5. Les lignes invalides sont ignorées.
4. Calculs du lot	foreachRDD : reduceByKey par catégorie, somme des notes, word count	Mêmes calculs que le Batch, appliqués seulement aux avis du lot.
5. Alerte	filter(note <= 2) puis collect()	Les avis négatifs du lot sont affichés avec « ALERTE avis negatif ».
6. Cumul	Map.merge(clé, valeur, somme)	Les résultats du lot sont ajoutés aux totaux depuis le démarrage.
7. Sortie	affichage console + output/streaming_resultats.txt	À chaque lot on affiche le nombre d'avis reçus, le total cumulé, les moyennes et le top des mots.

5.3 Démonstration de l'arrivée progressive des données

Procédure : (1) lancer *EventSender* (il attend Spark) ; (2) lancer *StreamingAvis* ; (3) observer la console : un avis est envoyé toutes les 2 secondes, et toutes les 5 secondes Spark affiche un lot contenant 2 à 3 nouveaux avis. Le total cumulé augmente à chaque lot. Le regroupement exact des avis par lot dépend de l'horloge de Spark et peut varier d'une exécution à l'autre ; **l'état final, lui, est toujours le même**.

Résultat final attendu après les 6 événements :

Catégorie	Avis reçus	Somme des notes	Note moyenne
Restaurant	2	5 + 1 = 6	3,00
Hotel	2	2 + 5 = 7	3,50
Application	2	4 + 1 = 5	2,50
Total	**6**	**18**	**3,00**

Trois alertes doivent apparaître (événements 2, 4 et 6). Le top des mots cumulé doit mettre en tête *chambre*, *application* et *plats* (2 occurrences chacun), puis les autres mots à égalité.

Test manuel pour la démonstration : lancer *EventSender* avec l'argument *manuel* et taper *Hotel;1;Chambre sale*. Dans les 5 secondes suivantes : le compteur Hotel augmente de 1, la note moyenne Hotel est recalculée et une alerte apparaît. Une ligne invalide (ex. *Hotel;9;test*) est ignorée.

6. Questions de compréhension sur notre projet

Q1. Pourquoi notre sujet a-t-il besoin d'un traitement Batch ? Quel calcul serait difficile ou peu utile événement par événement ?

Le Batch sert à analyser **tout l'historique** : classement des mots les plus fréquents, note moyenne par catégorie, répartition globale des sentiments. Ces résultats n'ont de sens que sur l'ensemble des avis. Les recalculer à chaque nouvel avis serait coûteux et peu utile, car un seul avis change très peu une statistique calculée sur des milliers de lignes.

Q2. Pourquoi notre sujet a-t-il besoin d'un traitement Streaming ? Quelle information veut-on actualiser sans attendre la fin d'une collecte ?

Les avis arrivent en permanence et un avis négatif doit être vu **tout de suite** pour que l'entreprise puisse réagir (contacter le client, corriger un bug). Le Streaming met à jour le nombre d'avis, la note moyenne par catégorie et déclenche l'alerte sans attendre qu'un fichier complet soit disponible.

Q3. Quelle est la différence entre les données historiques du Batch et les événements du Streaming ?

Les données du Batch sont un **fichier fini** (data/avis.csv, 20 avis) lu entièrement, avec un en-tête et 5 champs (id, date, categorie, note, commentaire). Les événements du Streaming sont des **messages courts** (categorie;note;commentaire) qui arrivent un par un par le socket, sans fin connue, et sont traités par micro-lots de 5 secondes. Le Streaming ne relit jamais le fichier historique.

Q4. Donnez un exemple précis d'entrée et le résultat attendu pour chaque traitement.

Batch : entrée = les 7 lignes de la catégorie Hotel du fichier ; résultat = « Hotel : 7 avis, note moyenne 3,43 » (notes 2, 5, 4, 3, 1, 5, 4 : somme 24, 24 / 7 = 3,43).
Streaming : entrée = *Hotel;2;Chambre sale et bruit* ; résultat = « Hotel : 1 avis, note moyenne 2,00 » et une ligne « ALERTE avis negatif ». Si l'on envoie ensuite *Hotel;5;Personnel aimable et chambre propre* : « Hotel : 2 avis, note moyenne 3,50 ».

Q5. Comment vérifier les résultats avec un petit jeu de données dont on connaît le résultat ?

Nous utilisons un jeu de 20 avis (Batch) et 6 événements (Streaming) assez petits pour être recalculés à la main ou avec un second programme indépendant. On compare ensuite chaque valeur affichée par Spark au tableau attendu (sections 4.3 et 5.3) et on vérifie des contrôles de cohérence (6 + 7 + 7 = 20 ; 11 + 3 + 6 = 20 ; total des événements = 6).

Q6. Quelles parties du code du TP peut-on réutiliser, et lesquelles faut-il adapter ? Expliquer chaque adaptation.

Le code du TP fournit le squelette ; nous l'adaptons à l'analyse d'avis :

Partie du TP	Réutilisée ou adaptée	Explication de l'adaptation
Word count (flatMap, mapToPair, reduceByKey)	Réutilisée et adaptée	Le découpage par espaces est remplacé par un découpage en mots propres (minuscules, lettres seulement, 3 lettres minimum, mots vides retirés) dans la classe commune *Texte*.
Lecture d'un fichier texte	Adaptée	Le fichier est un CSV avec en-tête : on retire la 1re ligne, on découpe en 5 colonnes avec *split(",", 5)* et on ignore les lignes mal formées.
Clé du reduceByKey	Adaptée	La clé n'est plus le mot mais la catégorie (nombre d'avis) ou le sentiment ; pour la moyenne on additionne un couple (somme des notes, nombre d'avis).
socketTextStream (port 9999)	Réutilisée	Même principe que le TP. Ajout du programme *EventSender* qui remplace *nc -lk 9999* et fonctionne sous Windows.

Partie du TP	Réutilisée ou adaptée	Explication de l'adaptation
Micro-lots Streaming	Adaptée	Ajout d'une validation des lignes (3 champs, note de 1 à 5) et d'un **cumul** entre les lots pour avoir des totaux depuis le démarrage.
Affichage console	Adaptée	Affichage d'un rapport (moyennes, top mots) et écriture dans *output/* avec *Files.write*. Ajout de l'alerte sur les notes inférieures ou égales à 2.

Q7. Si l'on utilise un code d'exemple ou une ressource externe : peut-on expliquer chaque étape, identifier nos modifications et justifier les résultats ?

Oui. Le point de départ est le word count du TP. Chaque étape est expliquée dans les tableaux des sections 4.2 et 5.2. Nos modifications sont listées dans le tableau ci-dessus. Les résultats sont justifiés par la vérification de la section 4.3 et 5.3 : ils sont recalculables à la main sur le jeu de test, et les contrôles de cohérence sont respectés. **[À personnaliser : si une partie du code vient d'une ressource externe, la citer ici.]**

7. Captures, explication personnelle et difficultés

7.1 Captures d'écran commentées (à insérer)

Les captures doivent être prises sur **votre propre exécution**. Remplacez chaque cadre par la capture correspondante et gardez le commentaire en dessous.

Capture 1 : exécution de BatchAvis
Rapport complet dans la console (20 avis, moyenne 3,35, top 10 des mots).

Commentaire : les valeurs correspondent au tableau attendu de la section 4.3, donc le Batch est validé.

Capture 2 : fichier data/avis.csv et fichier output/batch_resultats.txt

Commentaire : preuve de l'entrée utilisée et du résultat enregistré.

Capture 3 : EventSender et StreamingAvis côte à côte, plusieurs lots

Commentaire : on voit les avis envoyés un par un et le total cumulé qui augmente d'un lot à l'autre (arrivée progressive).

Capture 4 : lot avec « ALERTE avis negatif » et état final

Commentaire : l'état final doit afficher 6 avis, Restaurant 3,00, Hotel 3,50, Application 2,50 (section 5.3).

7.2 Explication personnelle des étapes

[À reformuler avec vos propres mots, par exemple :] Nous avons d'abord compris la différence entre Batch et Streaming grâce à l'exemple du word count. Pour le Batch, nous avons lu le CSV en RDD, transformé chaque avis en couple (clé, valeur) puis regroupé avec reduceByKey. Pour le Streaming, nous avons reçu les avis par socket, traité chaque micro-lot avec les mêmes opérations, puis cumulé les résultats pour avoir un total depuis le démarrage. Nous avons utilisé la même classe *Texte* dans les deux traitements afin que les résultats soient comparables.

7.3 Difficultés rencontrées (exemples à adapter à votre expérience)

• **Version de Java :** avec Java 17, Spark demande des options *--add-opens* dans la configuration d'exécution ; sans elles le programme s'arrête avec une erreur d'accès.

• **Chemin du fichier :** si le répertoire de travail n'est pas la racine du projet, *data/avis.csv* est introuvable.

• **Ordre de lancement :** EventSender doit être lancé avant StreamingAvis, sinon Spark ne trouve pas le serveur sur le port 9999.

• **Cumul entre les lots :** comprendre qu'un micro-lot ne contient que les nouveaux avis, d'où le besoin d'un état cumulé.

• **Qualité du texte :** sans retirer les mots vides (« les », « des », « pour »...), le top des mots ne montre rien d'utile.

• **Résultats variables :** le regroupement des avis par lot change d'une exécution à l'autre, seul le total final est stable.

7.4 Conclusion

Le projet montre les deux modes : le Batch donne une vue d'ensemble fiable sur l'historique, le Streaming donne une vue immédiate sur les nouveaux avis et permet de réagir vite. Les deux sont complémentaires pour le suivi de la satisfaction client.

Annexe : fichier data/avis.csv complet (jeu de test Batch)

id	date	categorie	note	commentaire
1	2026-10-01	Restaurant	5	Service rapide et plats delicieux
2	2026-10-01	Restaurant	4	Plats delicieux mais attente un peu longue
3	2026-10-02	Hotel	2	Chambre sale et personnel pas aimable
4	2026-10-02	Hotel	5	Chambre propre et personnel tres aimable
5	2026-10-03	Application	1	Application lente et bugs partout
6	2026-10-03	Application	4	Interface claire et application rapide
7	2026-10-04	Restaurant	2	Service lent et plats froids
8	2026-10-04	Hotel	4	Personnel aimable et chambre confortable
9	2026-10-05	Application	3	Application correcte mais interface confuse
10	2026-10-05	Restaurant	5	Service rapide et personnel aimable
11	2026-10-06	Hotel	3	Chambre correcte mais bruit la nuit
12	2026-10-06	Application	2	Bugs frequents et application lente
13	2026-10-07	Restaurant	4	Plats delicieux et service rapide
14	2026-10-07	Hotel	1	Chambre sale et bruit toute la nuit
15	2026-10-08	Application	5	Application rapide et interface claire
16	2026-10-08	Restaurant	3	Service correct mais plats froids
17	2026-10-09	Hotel	5	Personnel aimable et chambre propre
18	2026-10-09	Application	4	Interface claire et application utile
19	2026-10-10	Restaurant	1	Attente longue et plats froids
20	2026-10-10	Hotel	4	Chambre propre et service rapide

Structure du projet Maven : *pom.xml*, *data/avis.csv*, *src/main/java/com/projet/avis/* : *Texte.java* (outils communs), *BatchAvis.java*, *StreamingAvis.java*, *EventSender.java*.